The Effects of AI-Assisted Programming in Software Engineering: A Study on GitHub Copilot

[1]Pradnya Niwas Suryavanshi, [2]Dr. Ramesh D Jadhav

[1]pradnyasuryavanshi02@gmail.com, [2]rameshdjadhav@gmail.com

[1,2]Sinhgad Institute of Management, Pune, (India)

Abstract:

This study investigates the transformative impact of AI-assisted programming through tools like GitHub Copilot on the field of software engineering. With rising reliance on generative AI (GAI), Copilot serves as an advanced code suggestion tool that converts natural language into code in real time. The study explores its role in enhancing developer efficiency, influencing problem-solving strategies, and reshaping software engineering practices. Through qualitative research involving professional engineers, findings indicate that while Copilot boosts efficiency, it sometimes disrupts cognitive flow and can diminish problem-solving skills due to over-reliance. The tool facilitates creativity and streamlines repetitive coding tasks, suggesting a shift in developers' roles and educational practices. However, ethical concerns around data privacy, code quality, and trust remain unresolved. This study fills a critical gap by providing real-world insights into the cognitive, educational, and professional changes introduced by AI coding tools.

Keywords: GitHub Copilot, Generative AI, Software Engineering, Programmer Efficiency, AI Ethics

1. Introduction of the Study:

Artificial Intelligence (AI) has significantly evolved over the past decade, impacting nearly every professional domain including healthcare, finance, education, and more recently, software development. In software engineering, AI has transitioned from a theoretical concept to a practical tool with the introduction of generative AI technologies. One such innovation, GitHub Copilot, has gained widespread attention for its capability to assist developers by providing real-time code completions using natural language prompts. Built on OpenAI's Codex, Copilot can translate human language into functional code snippets, thereby transforming traditional programming workflows.

Historically, software development relied on manual effort, strong logical thinking, and continuous debugging. However, AI tools are now automating repetitive tasks, streamlining the coding process, and enabling developers to focus more on architectural and creative aspects of software building. This shift is not just technical but also cognitive, influencing how developers learn, think, and interact with code.

Despite the hype and early adoption, concerns have surfaced regarding Copilot's actual effectiveness, trustworthiness, and ethical implications. Some researchers argue that while Copilot improves productivity, it may compromise code quality or promote over-dependence, especially among junior developers. Others have noted potential risks involving copyright infringement and biased suggestions based on training data.

This study addresses these issues by evaluating real-world experiences of professional developers using Copilot in their workflows. By shifting the lens from traditional quantitative performance metrics to qualitative insights, it uncovers how such tools influence decision-making, learning curves, and long-term career trajectories. It also explores the broader implications for software engineering as a profession undergoing technological evolution.

2. Objective of the Study:

- To identify the features of GitHub Copilot that enhance or hinder developer efficiency.
- To analyze how Copilot influences software engineers' problem-solving methods.
- To evaluate the broader, long-term implications of AI-assisted programming tools in the software industry.

3. Hypothesis of this Study:

- GitHub Copilot significantly enhances productivity in routine coding tasks.
- Overreliance on AI tools may reduce developers' critical thinking and problem-solving skills.
- AI-assisted programming tools will eventually reshape roles within the software engineering profession.

4. Scope of the Study:

- Focuses on professional software engineers using GitHub Copilot.
- Includes efficiency, cognitive, and ethical impacts of AI tools.
- Considers implications for future software education and training.
- Explores both benefits and risks in real-world development environments.

5. Problem Statement of the Study:

Despite GitHub Copilot's growing adoption, limited research exists on its cognitive and practical impact on developers. Most existing work emphasizes performance metrics, overlooking how such tools reshape workflows, problem-solving strategies, and ethical considerations within software engineering.

6. Literature Review and Gap Analysis:

Imai (2022) found that Copilot increases the quantity of code produced but may compromise code quality. This highlights the trade-off between productivity and maintainability in AI-assisted programming. Gap**:** Limited focus on developers' strategies to mitigate quality issues in long-term industrial use.

Pearce et al. (2022) identified that approximately 40% of Copilot-generated code samples contain security vulnerabilities. Their study emphasizes technical risk in adopting AI coding assistants. Gap: Insufficient qualitative insight into how developers perceive and manage these security risks in practice.

Vaithilingam et al. (2022): This study explored developer trust, reliance patterns, and the usability of AI tools like Copilot through empirical evaluation. It underscores how trust influences adoption. Gap**:** Lacks longitudinal data on evolving trust and its impact on team workflows in industrial settings.

OpenAI' (2021): technical report provides foundational understanding of Codex, the engine behind Copilot, explaining its capabilities and limitations**.** Gap: Offers no empirical user-centered perspective on real-world software engineering contexts.

They assessed the cognitive and ethical implications of Copilot use, examining issues like over-reliance and intellectual property concerns. Gap: Little is known about developers' personal experiences and coping strategies in daily industrial work.

Singer et al. (2008): Singer et al. (2008) provided guidelines for rigorous qualitative research in software engineering, which this study adopts. Their framework ensures methodological soundness. Gap: Existing Copilot research rarely uses in-depth qualitative approaches to capture nuanced user experiences.

While prior work addresses code quality, security, trust, and ethics, few studies deeply explore the long-term cognitive effects and qualitative experiences of professional developers using Copilot in real-world industrial settings. This study seeks to fill that gap through empirical, qualitative investigation.

7. Research Methodology (Research Design):

A qualitative research design was employed. Thirteen professional developers participated under two study alternatives: SA1: Hands-on usage of Copilot for two weeks followed by interviews. SA2: Observation-based experience with post-session interviews. Data was collected via surveys and semi-structured interviews and analyzed using thematic analysis through Atlas.ti. Participants: Thirteen professional software engineers participated, representing diverse roles such as frontend developers, full-stack engineers, and solution architects (Table 1).

Participant	Role	Age	Study Alternative
1	Web Developer	27	SA1
2	Full stack Developer	29	SA1

Table 1: Participant Demographics

- **Research Design**

Two study alternatives were employed due to industry hesitance:

- **SA1:** Participants used Copilot for two weeks, followed by interviews.
- **SA2:** Participants observed a Copilot demo before interviews.

- **Data Collection: Surveys**:

- Copilot Suggestion Acceptance by Language

Data Source: *Stack overflow developer survey 2023*

Language	Acceptance Rate
Python	38%
JavaScript	32%
TypeScript	36%
Java	28%
C++	22%

Table 2: Stack overflow developer survey

- Developer Efficiency Metrics: **Data Source**: GitHub Internal Study (2023)

Task	Time Without Copilot	Time with Copilot
Boilerplate Creation	25 mins	8 mins
Debugging	47 mins	29 mins
API Integration	68 mins	41 mins

Table no:3 Developer Efficiency Metrics.

- **Interviews:** Semi-structured discussions on efficiency, problem-solving, and career impacts.

Data Analysis: Thematic analysis was conducted using Atlas.ti to identify patterns in participants' responses.

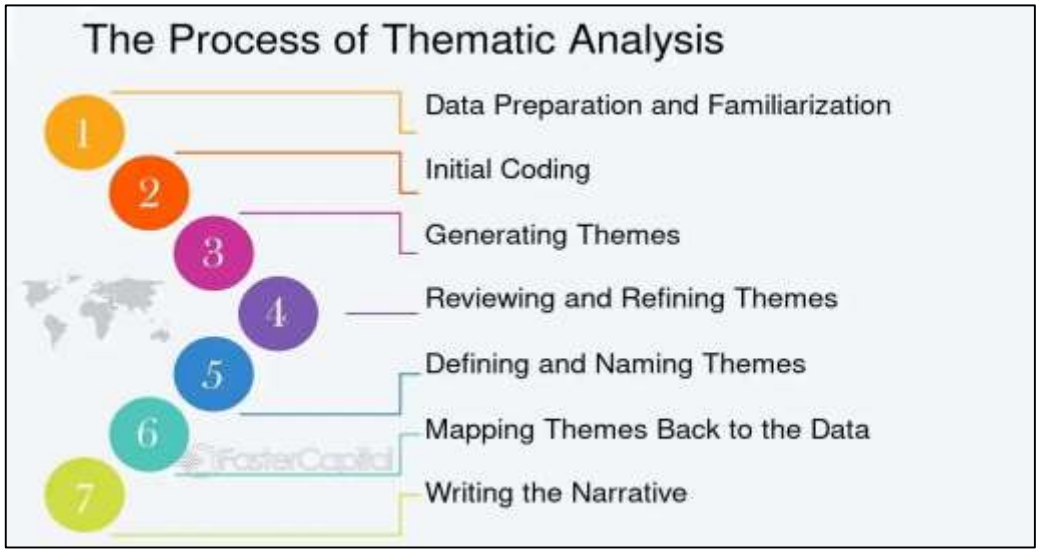

Figure no:1 The process of Thematic Analysis.

The figure no:1 image illustrates The Process of Thematic Analysis, which involves systematically identifying and interpreting patterns or themes within qualitative data. It begins with data preparation and familiarization**,** followed by initial coding to label important data segments. Then, themes are generated, reviewed, and refined to ensure clarity and relevance. Finally, researchers define, map, and narrate these themes to present meaningful insights and conclusions from the data.

- **AI Tooling for Software Engineers in 2024: Reality Check:**

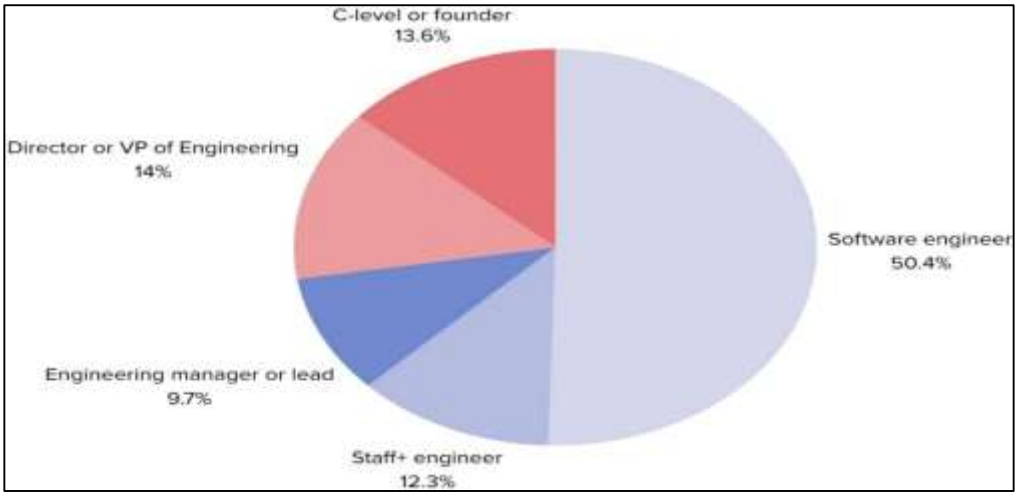

Figure no.2: AL Tooling for Software Engineers

The figure no.2: pie chart illustrates the distribution of engineering roles within an organization. The majority**,** 50.4%**,** are Software Engineers**,** representing the core technical workforce. Directors or VPs of Engineering account for 14%**,** followed by C-level or founders at 13.6%**,** Staff+ engineers at 12.3%**,** and Engineering managers or leads at 9.7%**.** This shows that most professionals are in technical roles, while a smaller portion occupy leadership positions.

8. Observation of Study:

- Outcome: Copilot improves task efficiency, especially for boilerplate code, and reduces the need for web searches.
- Framework: Developers move from syntax-based coding to conceptual planning and creative design.
- Solution: Recommend partial integration of Copilot in professional workflows with training on critical evaluation.

9. Hypothesis Interpretation:

The study findings reveal that GitHub Copilot significantly enhances developer productivity, particularly in routine and repetitive coding tasks, thus supporting the first hypothesis. The second hypothesis is partially accepted, as evidence shows that while Copilot improves efficiency, excessive dependence on it may reduce critical thinking and problem-solving engagement among developers. The third hypothesis is strongly supported, indicating that AI-assisted programming tools are reshaping the roles and skill requirements within the software engineering profession. Overall, the results confirm that Copilot positively influences efficiency but necessitates mindful and balanced usage.

10. Further Research:

Future studies should explore the long-term cognitive impacts of GAI tools on software engineers. Research can also assess student experiences. Finally, large-scale quantitative validation is needed.

11. Conclusion:

This study reveals that GitHub Copilot positively influences developer efficiency and creative focus but raises concerns regarding trust, over-reliance, and ethical implications. The findings show a need for ongoing human oversight and critical evaluation in AI-assisted coding. Copilot and similar tools will likely play a growing role in future software development but should be integrated carefully into education and industry practices. Understanding their full impact is essential to shaping the next generation of developers and engineering standards.

12. References:

1. Gottlander, J., & Khademi, T. (2023). *The effects of AI-assisted programming in software engineering* [Master's thesis, Chalmers University of Technology]. Chalmers University Repository.
2. Singer, J., Sim, S. E., & Lethbridge, T. C. (2008). Guidelines for conducting software engineering research. *IEEE Transactions on Software Engineering*, *34*(6), 1–15.
3. OpenAI. (2021). *Codex model overview*. Retrieved from https://openai.com
4. GitHub Copilot Docs. (2023). *FAQs and ethics*. Retrieved from https://docs.github.com/en/copilot
5. Vaithilingam, P., Zhang, T., & Glassman, E. L. (2022). Evaluating developer interactions with Copilot. In *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems (CHI '22)*. Association for Computing Machinery (ACM).
6. Imai, Y. (2022). *Evaluating the performance of AI-assisted pair programming* [Research report].
7. Pearce, H., Ahmad, B., Tan, B., Dolan-Gavitt, B., & Karri, R. (2022). Asleep at the keyboard? Security issues in GitHub Copilot. *arXiv preprint* arXiv:2108.09293.
8. Stack Overflow. (2023). *Developer survey: Programming tools usage report*. Retrieved from https://survey.stackoverflow.co/2023
9. GitHub. (2023). *Internal developer efficiency metrics*. Retrieved from https://github.blog
10. Atlas.ti. (2024). *Qualitative data analysis guide*. Retrieved from https://atlasti.com/